

Public Key Infrastructure – Certificate

Cloud Security

CSE4059

Server Authentication

- **DNS** provides an address for *hanyang.ac.kr*
- A successful **TCP** connection shows that an endpoint at that address is reachable
- Neither step proves that the endpoint represents *hanyang.ac.kr*
 - For example, a fake DNS response could direct the browser to an attacker
 - And, the attacker can still accept the TCP connection
- The client therefore needs evidence tied to the requested service name
 - A name claimed by the responding endpoint is not sufficient
- **Server authentication**
 - Verifies that the connected endpoint represents the intended service

Integrity and Authenticity

- To evaluate evidence from a server, distinguish two questions
 - Where the data came from → Authenticity
 - Whether it was changed → Integrity
- **Authenticity**
 - Evidence that the data came from the claimed source
- **Integrity**
 - Detecting unauthorized changes to protected data
- Clients need a mechanism that connects the message to verifiable evidence from its source
 - Digital signatures provide such evidence using a public/private key pair

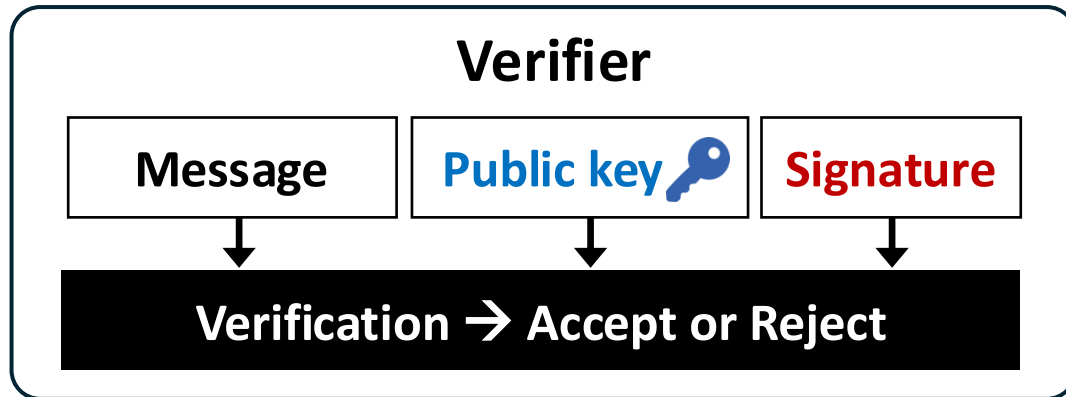
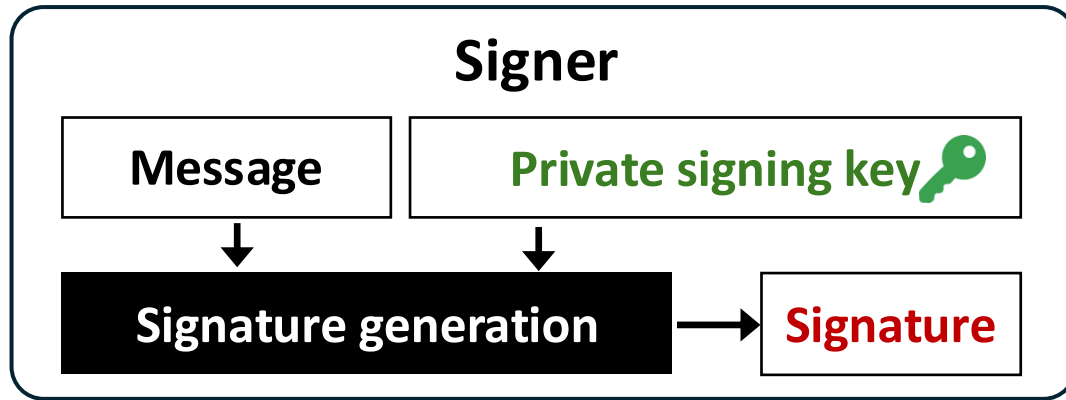
Key Pairs

- A key pair consists of a public key  and a mathematically related private key 
 - A key is a string of bits
 - Ex) public key: AAAAB3NzaC1yc2EBA3DAQADAAACAGDTBIBY3GKoZvftdYVnl...
- The public key  can be distributed to others
- The private key  must remain protected
- Knowing the public key should not reveal the private key or enable unauthorized signing
- The two keys have different cryptographic roles
 - We will use them for digital signature next

Digital Signatures

- Digital signature
 - A value generated from a message using a private signing key 
- The signer sends the message and signature together
- The receiver verifies the signature using the corresponding public key 
- A valid signature provides
 - Integrity: the signed message has not been modified
 - Authenticity with respect to the signing key: the signature was produced using the corresponding private key
 - Note that signing does not encrypt the message

Signature Generation and Verification



- Generation takes two inputs
 - The message and a private (signing) key
- Verification takes three inputs
 - The received message, its signature, and the public key
- The verifier tests whether the signature is valid for that message under that public key
 - Success: the three inputs satisfy the signature scheme's verification rule
 - Failure: if the message or signature changes, or the verifier selects an unrelated key

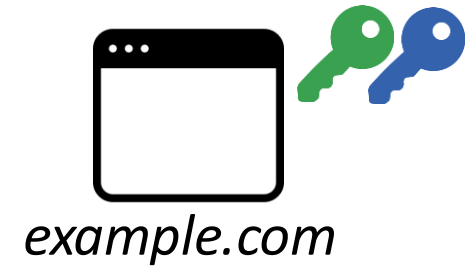
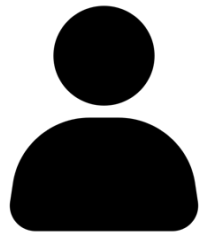
Verification Outcomes

- Verification takes three inputs
 - The received message, its signature, and the public key

Message	Signature	Verification key	Result
Original message	Original signature	Original public key	Pass
Changed message	Original signature	Original public key	Fail
Original message	Original signature	Unrelated public key	Fail
Attacker message	Attacker-generated signature (using attacker's private key)	Attacker public key	Pass

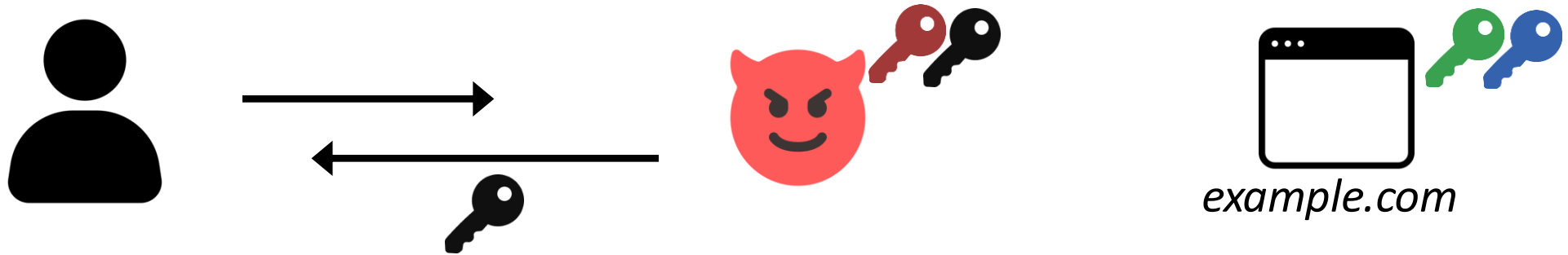
- Successful signature verification does not establish that the selected public key belongs to the intended service

Public Key Substitution



1. The client requests *example.com*

Public Key Substitution



1. The client requests *example.com*
2. An attacker supplies its own public key and labels it with that name

Public Key Substitution



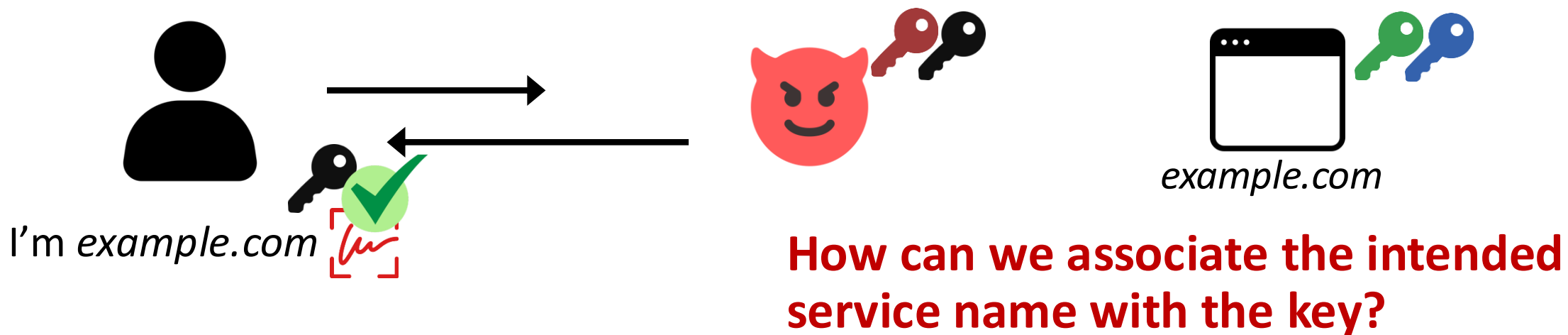
1. The client requests *example.com*
2. An attacker supplies its own public key and labels it with that name
 - The attacker signs a response using its matching private key

Public Key Substitution



1. The client requests *example.com*
2. An attacker supplies its own public key and labels it with that name
 - The attacker signs a response using its matching private key
3. The client's check succeeds if it uses the supplied key
 - The signature establishes consistency with the attacker's key, but the attached service name was never authenticated

Public Key Substitution



1. The client requests *example.com*
2. An attacker supplies its own public key and labels it with that name
 - The attacker signs a response using its matching private key
3. The client's check succeeds if it uses the supplied key
 - The signature establishes consistency with the attacker's key, but the attached service name was never authenticated

Certificates

- If a typical person hear the word “certificate”, they might mentally conjure up an image that looks something like this:
- IT/CS people, if they hear “certificates”, might first wonder if you're discussing staffers who've obtained professional credentials such as CCIE (Cisco Certified Internetwork Expert)



Certificates (cont'd)

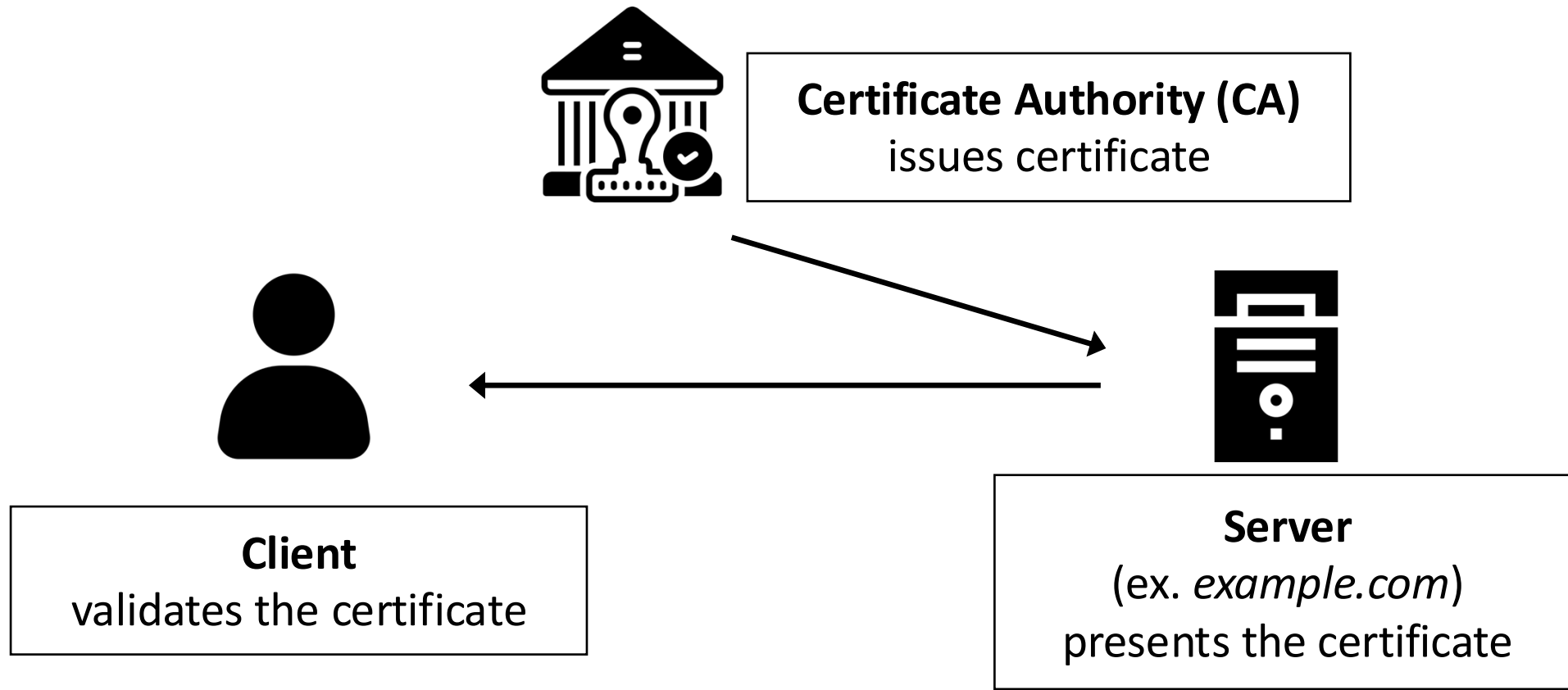
- A **certificate** is an electronic document that binds *identity* information to a *public key* through an *issuer's digital signature*
 - Digital certificates
 - Public key certificates
 - or shortly, cert(s)
- In the previous example, the identities are domain names like *example.com*
- Information in a certificate
 - States validity dates
 - Contains the identity's public key, not its private key
 - Includes the issuer's signature which protects the information against changes

Who Issues Certificates?

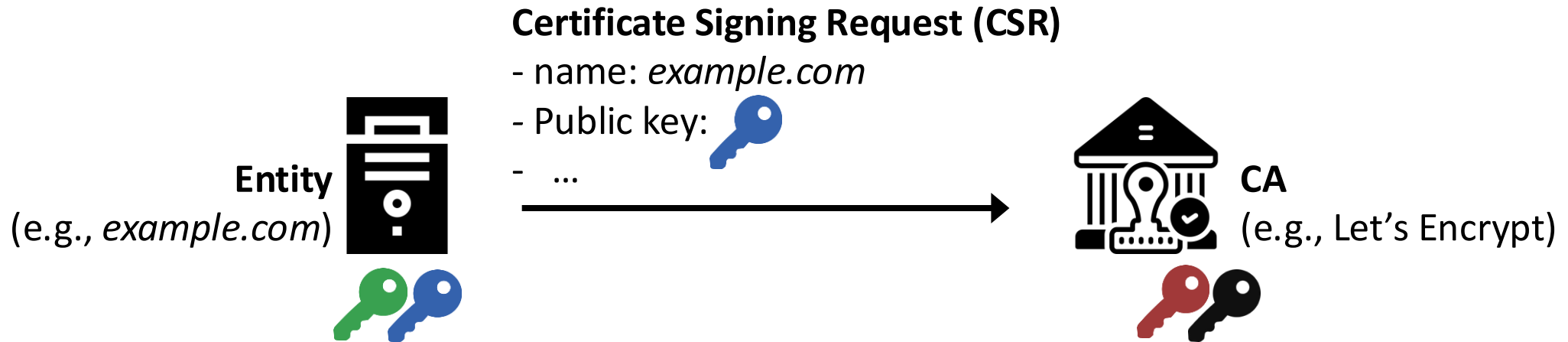
- A Certificate Authority (CA) is an issuer that checks issuance requests and signs the name-to-key statements it approves
 - A highly trusted third party (TTP)
 - Follows an issuance policy specifying which identity and key-related evidence the applicant must provide
 - Signs an approved certificate using its *own* private key (signature)
- Clients can verify the signature in the certificate using the CA's public key
- Various CAs
 - DigiCert
 - Let's Encrypt
 - Sectigo
 - ...

Public Key Infrastructure (PKI)

- **Public Key Infrastructure (PKI)** is the system of roles and processes used to issue, distribute, and manage certificates

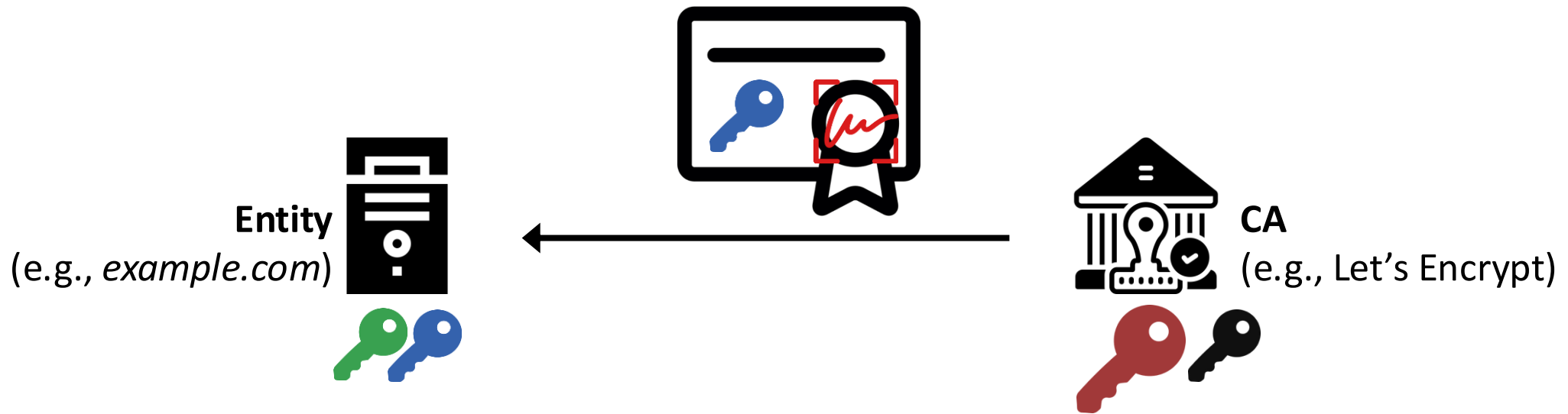


Certificate Issuance



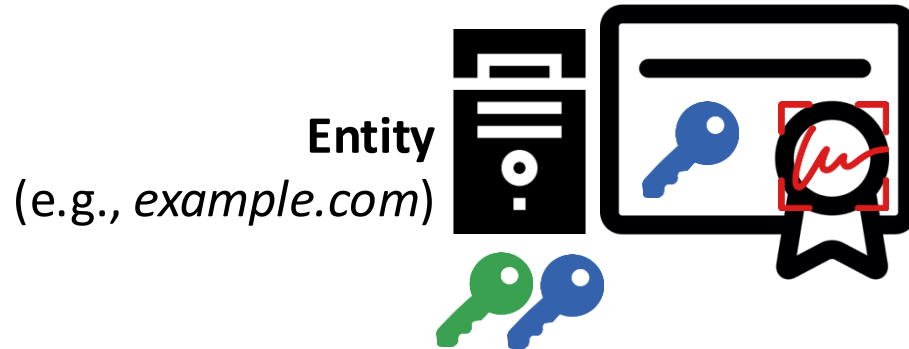
- An operator selects a key pair and requests a certificate for its public key and intended names (ex. domain names)

Certificate Issuance



- An operator selects a key pair and requests a certificate for its public key and intended names (ex. domain names)
- The CA checks the request and identity evidence before approving the binding and signing it with the CA's private key
 - And issues the certificate

Certificate Issuance



- An operator selects a key pair and requests a certificate for its public key and intended names (ex. domain names)
- The CA checks the request and identity evidence before approving the binding and signing it with the CA's private key
 - And issues the certificate
- The operator deploys the certificate

Certificate Requests and Private Keys

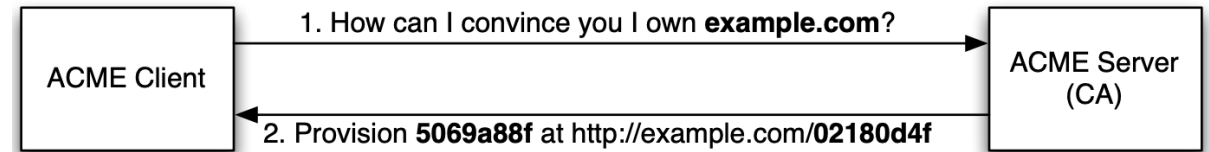
- A Certificate Signing Request (CSR) contains
 - Proposed names (e.g., domain names) and the applicant's public key
 - A signature generated using the corresponding private key
- The signature can demonstrate possession of the private key corresponding to that public key
- But an attacker can also generate a key pair and request another party's name
 - Key control alone cannot prove domain control
- The CA must therefore verify control of the requested identity separately before issuing the certificate

Domain Validation

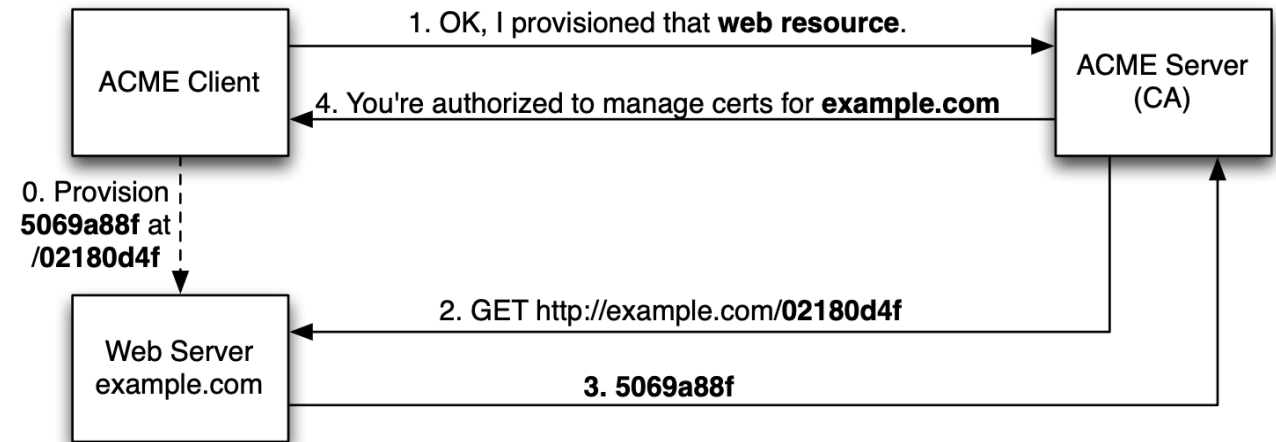
- Domain validation checks control of a domain
- For example, Let's Encrypt CA supplies a random challenge token to the applicant
 - HTTP-based method: Publish the required response to a specific path (e.g., *http://example.com/.well-known/validation-challenge/<token>*)
 - The CA retrieves it over HTTP on port 80 and checks its contents
 - DNS-based method: Publish the required value in a TXT record
 - The CA queries DNS and checks for a matching value
 - A correct response demonstrates control of the domain
- This verifies domain control, not application security or business legitimacy
- After successful validation, the CA issues a certificate to the applicant

ACME (Automatic Certificate Management Environment)

- A protocol for automating certificate issuance and renewal
 - Used by CAs such as Let's Encrypt
- The ACME client requests a certificate for one or more domain names
- The CA issues a challenge to verify control of the requested domain
- After successful validation, the CA issues the certificate



Challenge



Response

X.509 Certificate Structure

Version Number
Serial Number
Signature Algorithm ID
Issuer Name
Validity Period
Subject Name
Subject Public Key Info
Issuer Unique Identifier
Subject Unique Identifier
Extensions
Signature

- **Serial Number:** Uniquely identifies a certificate issued by a particular issuer
- **Signature Algorithm ID:** The algorithm used to create the signature
- **Issuer Name:** Entity that issued and signed the certificate
- **Validity Period:** The time interval during which the certificate may be accepted
- **Subject Name:** Entity associated with the certified public key
- **Subject Public Key Info:** Public key and algorithm information
- **Extensions:** Additional information and constraints
- **Signature:** Issuer's signature over the certificate body
- Issuer unique identifier? An optional legacy field used to disambiguate issuers that may have the same name; Rarely used in modern certificates

Subject Name and Subject Alternative Name

Version Number
Serial Number
Signature Algorithm ID
Issuer Name
Validity Period
Subject Name
Subject Public Key Info
Issuer Unique Identifier
Subject Unique Identifier
Extensions
Signature

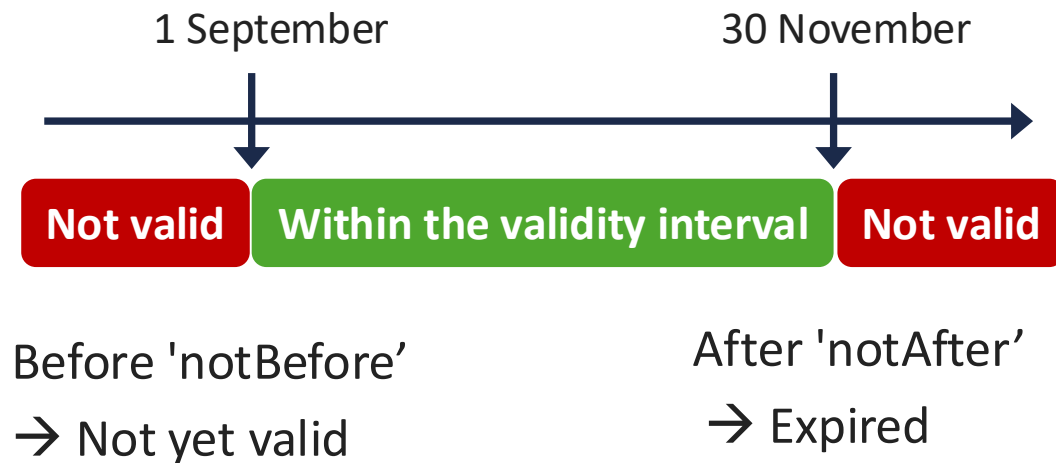
- Identity information can appear in the Subject field and in the Subject Alternative Name (SAN) extension.
- **Subject Name**
 - The entity associated with the certificate's public key
- **Subject Alternative Name (SAN)**
 - Included in the 'Extensions' field
 - List the identities associated with the certificate's public key
 - Allow one certificate to associate its public key with several names
 - Ex) *portal.example.com* and *files.example.com* in the SAN field
 - The same public key is certified for both names

Subject Public Key Info

Version Number
Serial Number
Signature Algorithm ID
Issuer Name
Validity Period
Subject Name
Subject Public Key Info
Issuer Unique Identifier
Subject Unique Identifier
Extensions
Signature

- **Subject Public Key Info** contains *Algorithm* and *subjectPublicKey*
- *Algorithm*
 - Identifies the public key algorithm
 - May include algorithm-specific parameters
 - Ex) RSA or elliptic-curve
- *subjectPublicKey*
 - Contains the subject's public key value
- The private key is NOT included in the certificate

Validity Period



- **Validity period?**

- The time interval during which the certificate may be accepted
- 'notBefore' and 'notAfter' define the start and end of that interval
- The client checks whether the validation time falls within the interval
- Expiry does not change the signed bytes → the issuer signature can still verify mathematically after the certificate expires

Certificate Signature

Version Number
Serial Number
Signature Algorithm ID
Issuer Name
Validity Period
Subject Name
Subject Public Key Info
Issuer Unique Identifier
Subject Unique Identifier
Extensions
Signature

- The certificate contains two signature-related fields
 - signatureAlgorithm: identifies the issuer's signature algorithm
 - signatureValue: the issuer's signature over the certificate body (*TBSCertificate*)
 - Contains the subject, public key, validity interval, and extensions
- The verifier uses the issuer's public key to verify the signature
- Changing any signed field invalidates the original signature
 - Ex) Replacing the subject public key causes signature verification to fail

Certificate Example

Certificate

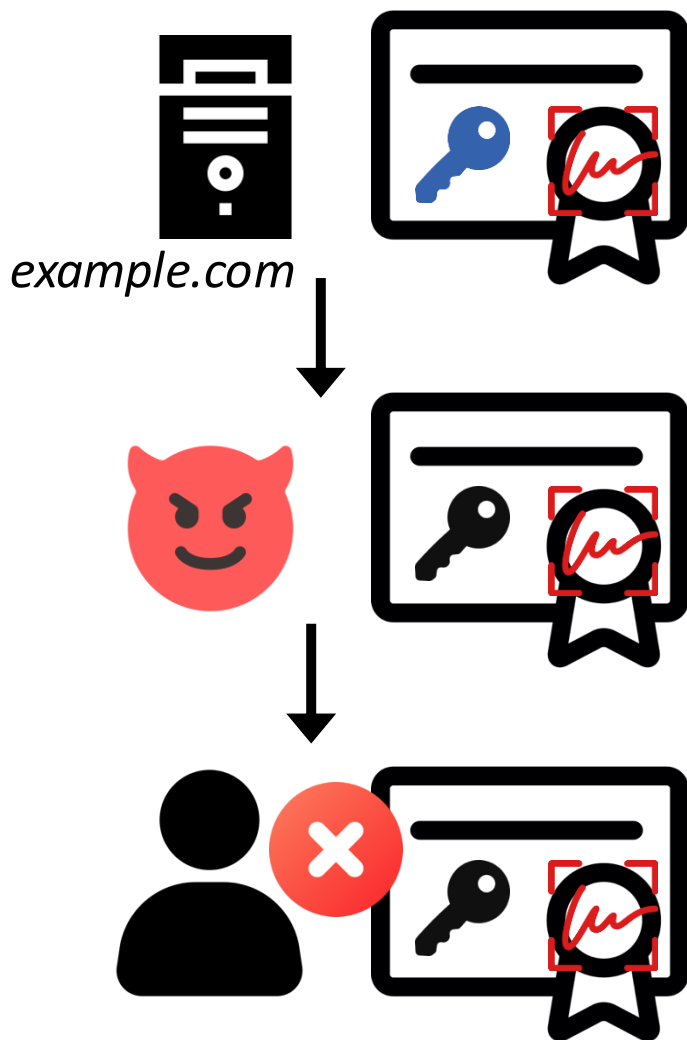
*.google.com		WR2	GTS Root R1
Subject Name			
Common Name	*.google.com		
Issuer Name			
Country	US		
Organization	Google Trust Services		
Common Name	WR2		
Validity		Public Key Info	
Not Before	Mon, 10 Aug 2026 08:37:35 GMT	Algorithm	Elliptic Curve
Not After	Mon, 02 Nov 2026 08:37:34 GMT	Key Size	256
		Public Value	04:FC:B7:1C:B3:56:A3:8B:48:44:A
Subject Alt Names			
DNS Name	*.google.com		
DNS Name	*.appengine.google.com		

Public Key Info	
Algorithm	Elliptic Curve
Key Size	256
Public Value	04:FC:B7:1C:B3:56:A3:8B:48:44:A9:D6:04:AD:68:2A:FE:FA:02:97:4A:70:5...

Certificate presented for *google.com*
(screenshot from Firefox)



Modified Certificates



- The original certificate binds *example.com* to the server key  and carries the issuer's signature 
- An attacker changes  to attacker key  but retains the original issuer signature
- The signed bytes now differ, so that signature no longer verifies against the modified body
- The attacker cannot transfer the approved binding to  by simply editing a field
 - An authorized change requires a newly issued signed certificate

Summary

- A digital signature links a message to a signing key (e.g., private key) and allows verification with the corresponding public key
- Public Key Infrastructure (PKI) is the system of roles and processes used to issue, distribute and manage certificates
- A certificate is a signed statement connecting identities (e.g., domain names) to a public key
 - Names, validity dates and extensions describe the signed statement
 - Changing protected fields invalidates the original signature
- A Certificate Authority (CA) issues approved certificates